

Technical Report

Introduction & Scope

Housing affordability remains a persistent economic and social challenge, especially in urban areas where rental costs can grow faster than incomes. This project addresses that problem using a data-driven web API that supports both operational data management and analytical insight generation. Rather than building a presentation-only interface, the project focuses on backend web-service engineering so that outputs can be reused across clients, scripts and decision-support workflows.

The implemented system provides complete CRUD functionality for regional and listing entities, then extends beyond baseline requirements with analytical endpoints for affordability interpretation. In addition to median rent and affordability ranking, the API includes scenario simulation, bedroom-based price trend analysis and a composite regional risk score. These features shift the system from simple record storage to an interpretable analytical service.

The scope also includes software engineering quality controls: scoped bearer-token authentication for write operations, strict schema validation and integrity handling, standardized error contracts, automated functional testing, endpoint benchmarking and reproducible dataset ingestion with provenance metadata. Together, these choices are intended to demonstrate both technical competence and independent design thinking .

Architecture & Stack Justification

The system uses a modular layered architecture to separate HTTP routing, persistence logic, analytical computation, and security concerns. At the entry layer, FastAPI (Ramírez, 2026) routers expose endpoint groups (``auth`` , ``regions`` , ``listings`` , ``analytics``). These delegate to dedicated CRUD and analytics modules, while shared schema models define request/response contracts and validation behavior. Security checks are applied through dependency-based middleware patterns on protected write routes. This structure reduces coupling, improves maintainability, and supports focused testing of each layer.

Python was selected as the implementation language due to its strong ecosystem for API development, data manipulation, and rapid iteration in analytical domains. FastAPI was chosen because it provides strict type-driven request validation, automatic OpenAPI generation and concise endpoint declaration. This enabled fast development whilst still keeping API contracts explicit and inspectable.

SQLAlchemy was chosen to model relational entities and enforce structured persistence patterns. It provides clear mapping between domain objects (`Region` , `Listing`) and SQL operations while preserving flexibility for query composition in analytics. SQLite (SQLite Consortium, 2026) was selected as the database engine for the scope due to portability, low setup overhead and sufficient relational capability for local reproducibility. This decision prioritized fast iteration and demonstrability and still supported foreign-key integrity and analytical query operations.

The architecture and stack choices were personal decisions guided by three criteria:

- (1) clarity under oral examination,
- (2) reliability of core data/analytics behavior,
- (3) ability to demonstrate progressive enhancement from baseline CRUD to advanced analytical functionality within project constraints.

Data Model Integrity & Security Design

The domain model is centered on two entities: `Region` and `Listing`. A `Region` stores geographical and socioeconomic context (`name` , `ons\_code` , `average\_income`), and a `Listing` stores market records (`price` , `bedrooms` , `listing\_type`) linked to a specific region. This one-to-many relationship reflects real housing datasets and enables affordability metrics that combine income and rent attributes.

To protect analytical correctness, integrity controls were implemented at multiple layers. At schema level, request payloads are validated using strict constraints (for example: positive prices/incomes, non-negative bedrooms, allowed listing types). At persistence level, foreign-key enforcement is explicitly enabled in SQLite connections, and listing-region references are constrained to prevent invalid relational state. Integrity exceptions from database operations are handled deterministically with rollback and translated to consistent API responses.

Security was designed around scoped bearer-token access. The API issues access tokens via ` /auth/token ` , and write endpoints (`POST` , `PUT` , `DELETE`) require authenticated principals with `write` scope. This allows clear role separation (e.g., read-only viewer vs write-capable admin) and demonstrates least-privilege controls within the scope. Read endpoints remain open to support straightforward analytics exploration and demonstration.

A standardized error contract was implemented across HTTP and validation failures. Instead of heterogeneous framework defaults, the API returns a consistent JSON envelope

with status code and message fields, plus validation detail payloads where relevant. This improves client predictability, simplifies testing assertions, and strengthens technical defensibility during Q&A.

Data Source Workflow

This project implements a reproducible multi-step dataset pipeline rather than relying exclusively on manual API input.

1. Dataset discovery and citation. The selected public dataset is Index of Private Housing Rental Prices from the Office for National Statistics (ONS, 2014). The CSV used in this workflow is <https://download.ons.gov.uk/downloads/datasets/index-private-housing-rental-prices/editions/time-series/versions/41.csv>. The dataset is licensed under Open Government License v3.0 (<https://www.nationalarchives.gov.uk/doc/open-government-licence/version/3/>), which permits academic use with attribution.
2. Download and explore. A dedicated script (`download_and_explore.py`) retrieves a selected dataset (by registry id or direct URL) and profiles its structure, including detected columns, row count, sample records, and null counts per field. This step is used to understand source format before transformation and identify potential quality issues early.
3. Merge and adaptation for internal schema. Because the ONS source is index-oriented rather than listing-level, the workflow applies a controlled merge step (`merge_ons_with_profiles.py`) using supplemental region profile data (`sample_region_profiles.csv`) to produce a staging dataset with internal fields (`region_name`, `region_ons_code`, `average_income`, `monthly_price`, `bedrooms`, `listing_type`).
4. Normalization for import. The staged dataset is transformed into canonical internal CSVs (regions and listings) using `prepare_external_dataset.py`. This script validates required fields, filters invalid records, and maps source fields into the project's relational schema contract.
5. Import into SQL database. The normalized files are loaded through `import_data.py`, which applies upsert behavior for regions (keyed by ONS code), validates listing rows, and inserts records with relational consistency.
6. Provenance metadata. Each import can generate an ingestion manifest containing UTC timestamp, file paths, SHA256 checksums, and import statistics (created, updated, skipped). This provides reproducibility and traceability evidence for both assessment and future extension.

This workflow demonstrates practical data engineering discipline: public-source citation, structural exploration, controlled transformation, and reproducible ingestion. It directly satisfies brief item (c) by using and explicitly citing a publicly available dataset.

Testing & Verification Approaches

Verification is implemented through an automated functional test suite and a separate performance benchmark script. Tests are written with Python ``unittest`` and FastAPI ``TestClient``, allowing endpoint-level validation against the running application stack.

The test collection covers:

- authentication and authorization behavior (``401`` for missing/invalid token, ``403`` for insufficient scope),
- schema validation and standardized error response shape (``422``),
- integrity/conflict behavior (``409``),
- list endpoint query behavior (filtering, sorting, pagination),
- analytics endpoint correctness (simulation, price trend, risk score, rankings),
- dataset workflow components (normalization, import behavior, provenance manifest structure).

The full test collection executes successfully (14 passing tests), providing concrete evidence of functional correctness across both core API and supporting scripts.

In addition to correctness tests, a benchmark utility (``benchmark_api.py``) measures endpoint latency for representative read operations and reports summary metrics (mean, p50, p95, max). While this is not a replacement for full-scale load testing, it provides reproducible baseline performance evidence and supports objective discussion of runtime behavior during oral examination.

Error handling verification is also explicit. The API uses a standardized JSON error envelope for HTTP and validation failures, enabling stable client contracts and deterministic assertions in tests. This reduces ambiguity and improves maintainability as features evolve.

Challenges & Lessons Learned

One of the main challenges was preserving data integrity while iterating quickly. Early versions exposed common backend risks such as weak constraint handling and inconsistent failure behavior under invalid writes. Resolving this required coordinated

changes across schema validation, database configuration, CRUD transaction handling, and router-level error translation. The key lesson was that analytics quality depends directly on the reliability of upstream data controls.

A second challenge was balancing simplicity and security. The project needed practical authentication that remained explainable and testable within the environment. The adopted scoped bearer-token model achieves this balance by demonstrating protected writes and role separation without introducing excessive infrastructure complexity. The lesson here was that security decisions should be proportional to scope while still reflecting real engineering principles (least privilege, explicit authorization checks, deterministic failure responses).

Finally, another problem involved analytical feature design. It was straightforward to expose single-value metrics, but significantly harder to produce endpoints that were both useful and defensible. The simulation, trend, and risk-score endpoints were introduced to move from descriptive outputs to comparative and decision-support analysis. A core lesson was to keep formulas transparent and interpretable so results can be justified under technical questioning.

Limitations & Future Work

Although the system is robust for the intended scope, several limitations remain.

First, the authentication implementation is intentionally lightweight. Scoped bearer tokens demonstrate access control clearly, but production-grade identity features (token revocation, key rotation policy, external identity provider integration, and audited session management) are not yet implemented.

Second, SQLite was selected for portability and reproducibility, not high-concurrency production deployment. As scale grows, a migration to a server-grade database and stronger operational controls (migrations, backup strategy, environment-specific configuration) would be appropriate.

Third, analytical models are currently deterministic and interpretable but not statistically calibrated against long historical datasets. In particular, risk-score weighting is heuristic rather than empirically tuned. Future work could incorporate calibration, uncertainty bounds, and historical validation to improve predictive reliability.

Fourth, data ingestion is script-driven. While reproducible and traceable, it is not yet scheduled or continuously synchronized with live data providers. A production extension would add automated ingestion jobs, versioned source snapshots, and monitoring for source schema drift.

Future development priorities are therefore:

- (1) security hardening
- (2) scale-oriented persistence and deployment improvements
- (3) empirically calibrated analytics
- (4) automated, monitored data-refresh workflows

Conclusion

This project delivers a data-driven housing analytics API that progresses well beyond baseline CRUD requirements. The implemented system combines modular backend architecture, relational integrity controls, scoped write authorization, standardized error contracts, and automated verification with benchmark evidence. It also includes an end-to-end data workflow for discovery, exploration, normalization, import, and provenance tracking.

From an engineering perspective, the project demonstrates iterative improvement from foundational API operations to advanced analytical capability (simulation, trend analysis, and composite risk scoring). From an assessment perspective, it provides concrete evidence of design reasoning, technical implementation quality, and reproducibility.

Overall, the final system is a strong example of practical web-service engineering applied to a socially relevant analytical domain, while still leaving clear pathways for production-level extension and methodological refinement.

Appendix & Gen AI Usage

Logs & Excrepts

This appendix contains representative excerpts/logs of GenAI-assisted (Microsoft Copilot as per university guidelines) interactions used during:

- architecture and design exploration:

- **API layer:**
 - REST controllers (or GraphQL resolvers).
 - Request validation, serialization, error mapping.
- **Domain / service layer:**
 - `ListingService`, `LocationService`, `AnalyticsService`, `AuthService`.
 - Encapsulate business rules: e.g., "affordability index = median rent / median income".
- **Data & integration layer:**
 - Repositories for DB access.
 - ONS / Land Registry ingestion jobs (ETL).
 - Caching (Redis) for hot aggregates.

Why this fits your use case

- **ONS / Land Registry integration:**
ETL jobs can be versioned and re-run as datasets update; domain layer stays stable.
- **Analytics:**
Domain services can precompute or lazily compute aggregates, then expose them via simple endpoints.
- **Extensibility:**
Easy to add new analytics (e.g., "rental volatility index") without breaking API contracts.

- Security model decisions, “Evaluate the suitability of this security model using bearer token issuance”

🔒 Security model suitability

Bearer token issuance at `/auth/token`

- Clean separation of authentication from business endpoints.
- Works well with FastAPI's dependency injection.

Scope-based authorization

- Simple but powerful:
 - `viewer` → read-only
 - `admin` → read/write
- Easy to extend later (e.g., `analytics:run`, `listings:write`, `regions:manage`).

Write routes require write scope

- Enforces least privilege.
- Prevents accidental data modification by read-only clients.

This model is enterprise-friendly without being over-engineered.

- data workflow planning and alternative exploration, “What alternatives exist other than my current approach to data workflow planning and which would you suggest?”

1. “Direct ingestion inside the API service”

What it is

- The API server downloads external datasets on startup or via an internal endpoint.
- Normalization and import happen inline in the same codebase and process.

Why teams choose it

- Fewer moving parts.
- Easy to trigger ingestion manually.

Why it's worse for your use case

- **Couples ingestion with API uptime** — a slow ONS dataset download can block your service.
- **Harder to test** — ingestion logic becomes entangled with request handlers.
- **No provenance** — you lose timestamps, checksums, and reproducibility.
- **No isolation** — a malformed dataset can crash your API.
- **Scaling pain** — ingestion and API traffic compete for CPU and I/O.

Moreover, GenAI was used in:

Dataset Discovery

data/dataset_registry.json (public dataset source + license) whereby I used GenAI to shortlist public housing datasets, then selected ONS-based source and documented license metadata

Download and Explore

‘scripts/download_and_explore.py’, whereby I used AI-assisted workflow to inspect dataset schema before transformation.”

Bibliography

ONS, 2014. *ONS*. [Online]

Available at: <https://www.ons.gov.uk/datasets/index-private-housing-rental-prices>
[Accessed 14 February 2014].

Ramírez, S., 2026. *FastAPI*. [Online]

Available at: <https://fastapi.tiangolo.com/>
[Accessed 24 February 2026].

SQLite Consortium, 2026. *SQLite*. [Online]

Available at: <https://sqlite.org/docs.html>
[Accessed 2 March 2026].

Links

- Public GitHub Repository:

[https://github.com/Yaseenali5/Web_Services_CW1](https://github.com/Yaseenali5/Web_Services_CW1)

- API Documentation:

https://github.com/Yaseenali5/Web_Services_CW1/blob/main/API_DOCUMENTATION.pdf

- Presentation Slides and Visual Assets:

<https://1drv.ms/p/c/611cd7da856f4f13/IQD7sQWFp1iGTIzIJ-tYfEvDAcT6MLksKwUbglqqbD1DjSc?e=AZRNgc>